SentinelOps

Self-Healing, Cost-Aware, Secure Kubernetes Platform

Complete Plan | Stack Matched to Parent Blueprint | No Card | Cloud Only | Design System

Ship fast. Stay secure. Self-heal.

OpenTofu + k3s + ArgoCD | Version 2.0 | 30 October 2026

1. Executive Summary

SentinelOps is a **self-healing, cost-aware, secure Kubernetes platform**. A developer pushes code to Git; the platform builds it, scans it, signs it, deploys it as a canary, watches it, rolls it back if it misbehaves, scales it, recovers it from failures, and reports what it costs. A custom web console ties everything together in one operator view.

The technology stack is **identical to the parent blueprint** (Advanced Cloud Computing Project Ideas, Part 2). Only the hosting changed, to meet two rules:

1. **No credit card anywhere.** Oracle Cloud is removed because its sign-up asks for a card.
2. **Nothing runs locally.** The kind cluster and "local k3s VM" fallbacks are removed. Everything runs in a cloud.

2. What Changed from the Parent File

Parent blueprint	This plan
Oracle Cloud Always Free (4 ARM OCPU, 24 GB RAM)	**Azure for Students** (no credit card, $100 credit, verified students) as Track A
Fallback: local k3s in a Linux VM or kind on a laptop	**GitHub Codespaces** (cloud-hosted, no card, free monthly quota) as Track B for anyone
OpenTofu provisions Oracle VCN, subnets, Ampere A1 VMs	OpenTofu provisions an **Azure VNet, NSG and Arm VMs** using the azurerm provider
State in Oracle Object Storage	State in **Azure Blob Storage**, or a private repo with OpenTofu native state encryption
kind 0.29.x for local dev clusters	**Removed** (violates the no-local rule)
Risk: Oracle ARM capacity unavailable	Risk: credit burn and regional vCPU quota (see section 7)
Everything else	**Unchanged**: same tools, same versions, same architecture, same screens, same 10-week plan

3. Problem Statement and Goals

Real-world problem	What SentinelOps does about it	Measurable result (target)
Slow, manual, error-prone deployments	GitOps and automated CI/CD	Commit to production under 5 min
Vulnerable code and images reach production	Trivy, Kyverno, image signing, SBOM	100% of insecure test deployments blocked
Bad releases cause outages	Canary releases with automatic rollback	Rollback under 60 s
Failures need a human at 3 a.m.	Self-healing, alert-driven remediation, chaos-tested	Recovery after pod or node kill under 30 s
Idle or oversized workloads waste money	KEDA scale-to-zero, right-sizing, OpenCost	Cost reduction of 30%+ in load test

4. Existing Apps vs SentinelOps

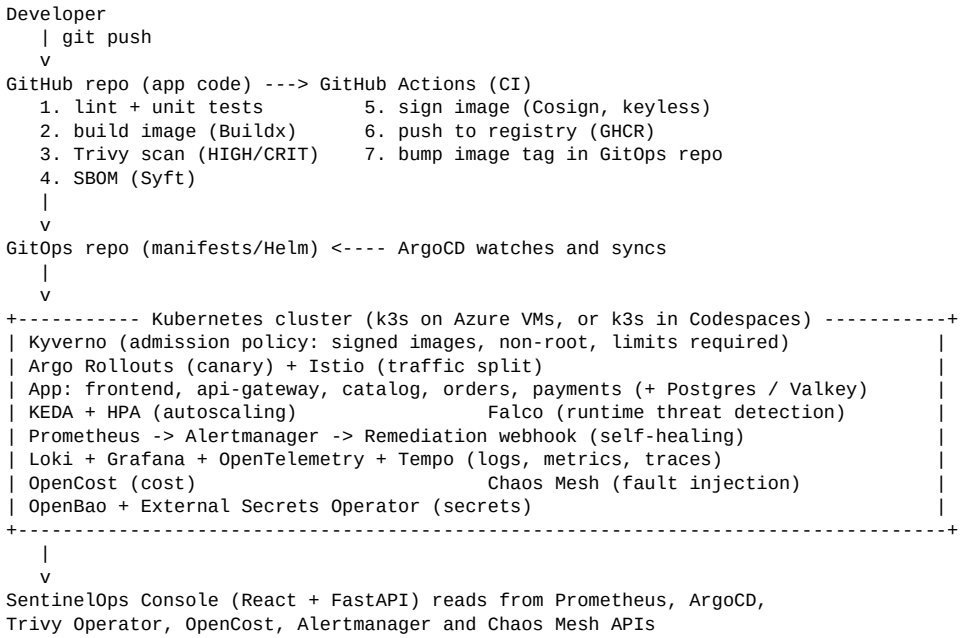
Most of SentinelOps is built from excellent open-source parts. Its value is **integration and proof**: one pipeline and one console where release safety, security gates, self-healing, chaos results and cost are connected and measured. The table describes **general strengths of each category**; vendor features change often, so verify before quoting them publicly.

Existing tool	What it does well	Typical gap SentinelOps addresses
Argo CD UI, Flux	GitOps sync and drift correction	Shows sync state only; no security findings, cost or chaos results in the same view
Rancher, OpenShift	Cluster management, some built-in security	Heavy platforms; not a transparent, reproducible, learn-by-building reference stack
Backstage	Developer portal and service catalog	Catalog focus; release verification and self-healing are left to plugins you assemble
Datadog, Grafana Cloud	Excellent observability	Paid at scale; observe and alert, but do not enforce policy or run safe-release gates
Komodor, Robusta	Kubernetes troubleshooting and alert enrichment	Focus on diagnosis; less on pre-deploy security gates, canary analysis and cost together
Harness, Spinnaker	Enterprise continuous delivery with verification	Large and costly to operate; not built around one free, reproducible reference stack
Kubecost / OpenCost UI	Cost allocation	Cost only; no link between savings, autoscaling events and releases
Trivy, Kyverno, Falco on their own	Strong security tools	Separate dashboards; nothing ties a blocked deploy to the release timeline

Novelty: what SentinelOps adds

1. **One console for the whole release lifecycle.** Canary progress, ArgoCD sync, vulnerabilities, policy blocks, runtime alerts, incidents, cost and chaos in eight connected screens.
2. **Security gates that are proven, not claimed.** Scripted attempts to deploy an unsigned image, a root container and a latest tag are all rejected by Kyverno, and the results are reported as a percentage.
3. **Safe-by-default releases.** Argo Rollouts with Istio traffic splitting and Prometheus analysis aborts a bad release without human action.
4. **Allow-listed self-healing with an audit trail.** The remediation webhook can only run approved actions with a restricted ServiceAccount, and every action is logged and shown in the console.
5. **Chaos as evidence.** Each experiment is a hypothesis, an expected result and a measured result, giving recruiters and reviewers real numbers.
6. **Cost tied to behaviour.** The same k6 load test runs with autoscaling off and on, and OpenCost reports the difference.
7. **Reproducible in the cloud for free, with no card.** One command per phase, everything as code, on free student or free-quota hosting.

5. System Architecture



Release flow: push, CI passes, image signed, manifest repo updated, ArgoCD syncs, Kyverno verifies the signature, Argo Rollouts shifts 10%, then 30%, then 60%, then 100% of traffic while Prometheus checks error rate and latency. Any breach triggers automatic rollback.

6. Technology Stack with Versions

Versions are the parent file's known-good picks as of mid-2026. Before starting, check each project's release page and pin to the latest stable patch of the same major/minor line. Pin everything: Helm chart versions, image tags and GitHub Action SHAs.

6.1 Infrastructure and cluster

Component	Version	Purpose	Cost
Ubuntu Server	24.04 LTS	Node OS	Free
OpenTofu	1.10.x	Infrastructure as Code (provisions Azure network and VMs)	Free, MPL-2.0
Ansible core	2.18.x	Node configuration, k3s install	Free
k3s	v1.33.x (Kubernetes 1.33)	Lightweight production-grade Kubernetes	Free
Helm	3.18.x	Package manager	Free
Kustomize	5.6.x	Environment overlays	Free
Azure for Students (changed)	$100 credit, 12 months	Real cloud hosting, no credit card	Free for verified students
GitHub Codespaces (changed)	120 core-hours and 15 GB per month	Cloud-hosted fallback for anyone, no card	Free quota

6.2 CI/CD and GitOps

Component	Version	Purpose
GitHub Actions	hosted runners (free for public repos)	CI pipeline
Docker Buildx	0.24.x	Multi-arch image builds (amd64 + arm64)
GHCR	n/a	Free container registry
ArgoCD	3.0.x or later 3.x	GitOps continuous delivery
Argo Rollouts	1.8.x	Canary and blue-green releases
Argo Image Updater (optional)	0.16.x	Auto-update image tags

6.3 Security

Component	Version	Purpose
Trivy	0.6x.x	Image, IaC and dependency scanning in CI
Trivy Operator	0.2x.x	Continuous in-cluster vulnerability reports
Syft	1.x	SBOM generation
Cosign (Sigstore)	2.5.x	Keyless image signing and verification
Kyverno	1.14.x	Admission policies (verify signatures, block root, require limits)
Falco	0.41.x	Runtime threat detection (eBPF driver)
OpenBao	2.x	Secrets management (open-source Vault fork)
External Secrets Operator	0.18.x	Syncs OpenBao secrets into Kubernetes
cert-manager	1.18.x	Automatic TLS certificates
Gitleaks	8.2x.x	Secret scanning in CI

6.4 Observability, scaling, cost, chaos

Component	Version	Purpose
kube-prometheus-stack (Helm chart)	7x.x (Prometheus 3.x, Alertmanager 0.28.x)	Metrics and alerting
Grafana	12.x	Dashboards
Loki + Promtail (or Grafana Alloy)	3.5.x	Log aggregation
Tempo	2.8.x	Distributed tracing
OpenTelemetry Collector	0.12x.x	Telemetry pipeline
KEDA	2.17.x	Event-driven autoscaling, scale-to-zero
metrics-server	0.7.x	Resource metrics for HPA
Goldilocks + VPA (recommend mode)	4.x / 1.4.x	Right-sizing recommendations
OpenCost	1.11x.x	Per-namespace and per-workload cost
Chaos Mesh	2.7.x	Pod kill, network delay, CPU stress experiments
k6	1.x	Load testing
Istio	1.26.x (ambient or sidecar mode)	mTLS, traffic splitting, retries

Service mesh note (from the parent): Linkerd no longer publishes free stable releases for new versions, so Istio is the safer open-source choice.

6.5 Application and console

Layer	Technology and version
Demo microservices	Go 1.24 (api-gateway, orders, payments), Python 3.12 + FastAPI 0.115 (catalog)
Frontend (demo shop)	React 19 + Vite 6 + TypeScript 5.8
Database	PostgreSQL 17 via CloudNativePG operator 1.26
Cache and queue	Valkey 8 (open-source Redis fork)
Console frontend	React 19, Vite 6, TypeScript 5.8, Tailwind CSS 4, shadcn/ui, Recharts 2.15, TanStack Query 5
Console backend	Python 3.12, FastAPI 0.115, httpx, Pydantic 2.11, kubernetes client 33
Console auth	Keycloak 26 (OIDC, roles: admin, developer, viewer)
Testing	pytest, Go test, Playwright 1.5x for UI, k6 for load

7. Free, No-Credit-Card, Cloud-Only Hosting

Every tool in section 6 is open source and free. The only cost question is **where the cluster runs**. Two tracks are provided; the software stack is identical on both.

7.1 Track A: Azure for Students (recommended, real cloud VMs)

Item	Detail
Eligibility	Verified university or college students; academic email verification
Card required	No
Credit	$100, valid 12 months, renewable each year while a student
Used for	Two Arm VMs (control plane and worker) in a VNet, as in the parent plan
Suggested sizes	Control plane and worker around 2 vCPU and 8 GB each (for example B2ps_v2); confirm SKU, price and regional quota in the Azure portal
Free-hours extra	The student offer also lists free monthly hours on small burstable VMs, which can host an optional third worker
Why Arm	The parent builds amd64 and arm64 images with Buildx, so Arm nodes work; x86 sizes are an equally valid choice

Credit-saving rules: deallocate VMs with `make down` after every session, set an Azure budget alert, run only the minimum stack first, and disable Istio and Tempo if memory is tight. The $100 credit will not cover two VMs running 24/7 for a year, so plan to run them only while working and recording demos.

7.2 Track B: GitHub Codespaces (anyone with a GitHub account)

Item	Detail
Card required	No; when the free quota is used up and no payment method is set, usage is blocked rather than billed
Free quota	120 core-hours and 15 GB storage per month on GitHub Free personal accounts
What it is	A cloud-hosted development VM, not your own machine
How the cluster runs	A dev container with Docker-in-Docker; k3s runs inside Docker using the k3d helper
Machine choice	A 4-core machine gives roughly 30 hours per month; choose the largest size your account offers and disable Istio and Tempo if memory is tight
Limits	Sessions stop when idle, and quota is shared across the whole month, so use it for build, test and demo sessions, not 24/7 hosting

k3d is the only tool added for Track B (to run k3s in a container). Track B skips the OpenTofu VM step because there are no VMs to create.

7.3 Everything else, and why it needs no card

Service	Free and card-free basis
GitHub, GitHub Actions, GHCR	Free for public repositories
Sigstore keyless signing	Uses GitHub OIDC identity; no key purchase
All CNCF and open-source tools	Open-source licences
OpenTofu state	Azure Blob Storage inside the student subscription, or a private repo using OpenTofu state encryption
DNS and TLS	Use the cluster's public IP with cert-manager and a free nip.io style hostname, or self-signed certificates for the demo

8. Repository Structure

The parent text says "three repositories" but lists four; this plan keeps all four.

```
sentinelops-app/            # application source + CI
  services/
    api-gateway/ (Go)  catalog/ (FastAPI)  orders/ (Go)
    payments/ (Go)     frontend/ (React)
  .github/workflows/ci.yml
  Dockerfile per service

sentinelops-gitops/         # desired cluster state (ArgoCD watches this)
  apps/                     # ArgoCD Application manifests (app-of-apps)
  platform/                 # argocd, kyverno, falco, keda, istio, monitoring...
  workloads/
    base/ + overlays/{dev,staging,prod}/
  policies/                 # Kyverno ClusterPolicies
  rollouts/                 # Argo Rollouts + AnalysisTemplates
  chaos/                    # Chaos Mesh experiments

sentinelops-infra/          # infrastructure
  tofu/                     # OpenTofu: Azure VNet, subnet, NSG, Arm VMs
  ansible/                  # k3s install, hardening, node config
  Makefile                  # make up / make down / make demo

sentinelops-console/        # custom web console
  frontend/ (React)
  backend/ (FastAPI)
```

9. Implementation Details

9.1 Provision infrastructure (OpenTofu + Ansible)

1. OpenTofu creates a resource group, VNet, public subnet, network security rules (22, 80, 443 and 6443, with 22 and 6443 limited to your IP) and 2 Arm VMs: one control plane, one worker.
2. OpenTofu outputs an Ansible inventory.
3. Ansible hardens the nodes (UFW, fail2ban, unattended upgrades, SSH key-only) and installs k3s with Traefik and the default load balancer disabled, because Istio ingress is used instead.
4. `make up` runs both and ends with a working kubeconfig; `make down` deallocates the VMs to save credit. State goes to Azure Blob Storage or an encrypted private repo.

9.2 Bootstrap GitOps

Install ArgoCD once with Helm, then apply a single root Application pointing to `sentinelops-gitops/apps`. Every platform component and workload is then installed by ArgoCD (app-of-apps pattern). Sync waves install CRDs and operators first: cert-manager and Kyverno, then Istio, monitoring and KEDA, then workloads.

9.3 CI pipeline (GitHub Actions)

```
# .github/workflows/ci.yml (outline)
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  test:        # unit tests per service, golangci-lint, ruff, eslint
  secrets:     # gitleaks
  build-scan:  # matrix per service
    steps:
      - docker buildx build (amd64+arm64) -> GHCR
      - trivy image --exit-code 1 --severity HIGH,CRITICAL
      - syft -> sbom.spdx.json (attach as attestation)
      - cosign sign --yes <image>@<digest>     # keyless via GitHub OIDC
  promote:     # only on main
    steps:
      - checkout sentinelops-gitops
      - update image digest in overlays/dev (kustomize edit set image)
      - open commit/PR -> ArgoCD syncs
```

Promotion to staging and prod is a pull request in the GitOps repo with a required approval, giving an audit trail.

9.4 The demo application

An e-commerce shop with five services: frontend, api-gateway, catalog, orders and payments. Orders publishes to a Valkey stream that payments consumes (this is what KEDA scales on). Each service exposes `/healthz`, `/readyz` and Prometheus `/metrics`. A feature flag injects errors or latency so you can deliberately ship a bad version and demonstrate automatic rollback.

9.5 Security layer

- **Kyverno policies:** require images signed by your GitHub OIDC identity; deny runAsRoot, privileged containers and latest tags; require CPU and memory requests and limits; require readOnlyRootFilesystem; restrict registries to GHCR; auto-generate default-deny NetworkPolicies per namespace.
- **Istio:** strict mTLS mesh-wide; AuthorizationPolicies so only api-gateway can call orders, and only orders can call payments.
- **Falco rules:** alert on shell spawned in a container, write below /etc, unexpected outbound connection, and reading service-account tokens. Events flow through Falcosidekick to Alertmanager and the console.

- **Secrets:** database passwords live in OpenBao; External Secrets Operator syncs them to Kubernetes. No secret ever appears in Git.
- **Proof for the demo:** deploy an unsigned image, a root container and a latest tag, and show all three rejected by Kyverno with clear messages.

9.6 Canary releases with automatic rollback

Each Deployment becomes an Argo Rollouts Rollout.

```
strategy:
  canary:
    trafficRouting: { istio: { virtualService: { name: orders-vs } } }
    steps:
      - setWeight: 10
      - pause: { duration: 60s }
      - analysis: { templates: [{ templateName: success-rate }] }
      - setWeight: 30
      - pause: { duration: 60s }
      - setWeight: 60
      - pause: { duration: 60s }
```

The AnalysisTemplate queries Prometheus: success rate must stay at or above 99% and p95 latency under 500 ms. A failed check aborts the rollout and shifts all traffic back to the stable version with no human action.

9.7 Observability and alert-driven self-healing

Every service exports metrics, structured JSON logs (Loki) and OpenTelemetry traces (Tempo), with a trace ID in each log line. Alertmanager rules cover high error rate, high latency, crash-looping pods, node not ready, growing queue backlog, certificate expiry and Falco critical events.

The remediation webhook (FastAPI, part of the console backend) receives Alertmanager webhooks and runs allow-listed actions with a restricted ServiceAccount: restart a crash-looping deployment, scale a service up one step, cordon and drain an unhealthy node, or trigger a rollback. Every action is audit-logged and shown in the console. Kubernetes provides the first layer of healing (probes, PodDisruptionBudgets, topology spread, replicas).

9.8 Autoscaling

Service	Scaler	Trigger	Range
frontend, api-gateway	HPA	CPU 60%	2 to 8 pods
catalog	HPA	requests per second (custom metric)	2 to 6 pods
payments	KEDA	Valkey stream length	0 to 10 pods (scale to zero when idle)
Dev and staging workloads	KEDA cron	scale to 0 outside working hours	0 to 2 pods

Goldilocks shows right-sizing recommendations; apply them in Git and show before and after in OpenCost.

9.9 Chaos engineering

1. **PodChaos:** kill a random orders pod during load. Expect no failed requests beyond the first few seconds.
2. **NetworkChaos:** 500 ms delay between orders and payments. Expect retries, circuit-breaking and a latency alert.
3. **StressChaos:** 90% CPU on a pod. Expect HPA to scale out.
4. **Node failure:** stop a worker VM (in Azure, deallocate it from the portal or CLI). Expect pods rescheduled within the recovery target.
5. **Bad release:** deploy a faulty version. Expect automatic rollback.

Write each as hypothesis, expected result and measured result.

9.10 Cost visibility

OpenCost reads Prometheus data and reports cost per namespace, deployment and label. Configure custom pricing (for example CPU at $0.03 per hour and RAM at $0.004 per GB-hour, close to public cloud on-demand prices) so numbers are realistic and clearly labelled as simulated. Run the same k6 test with autoscaling off (fixed 8 replicas) and on, and report the difference.

10. Custom Console: Screens and Features

The console is what recruiters see in a 60-second video. It does not replace Grafana or ArgoCD; it joins them into one operator view.

Backend API (FastAPI): `/api/overview`, `/api/deployments`, `/api/rollouts`, `/api/vulnerabilities`, `/api/policy-violations`, `/api/incidents`, `/api/remediations`, `/api/costs`, `/api/chaos/experiments`, `/api/chaos/run`. It talks to Prometheus, ArgoCD, Trivy Operator CRDs, OpenCost and Chaos Mesh. Login is through Keycloak (OIDC); roles control who can run chaos or approve rollbacks.

Screen	Content
1. Login	Keycloak sign-in (GitHub or username/password); role shown after login
2. Platform Overview	Cluster health, deploys today, open incidents, monthly cost, savings vs fixed, policy blocks, live request and error chart, recent events
3. Deployments and Releases	Services with version, environment, ArgoCD sync and rollout state; release detail with canary progress (10, 30, 60, 100%), live success rate and p95 vs thresholds, and Promote, Pause, Abort buttons (role restricted)
4. Security Center	Tabs: Vulnerabilities (Trivy Operator), Policy Violations (Kyverno), Runtime Alerts (Falco), SBOM and Signatures
5. Incidents and Self-Healing	Alertmanager alerts, the automated remediation that ran, whether it resolved the alert, time to recover, links to Grafana, Loki and Tempo
6. Cost and Capacity	Cost by namespace and service, requested vs used CPU and memory, top 5 Goldilocks savings, before and after autoscaling
7. Chaos Lab	Experiment catalog with hypothesis, blast radius, last result; Run button (admin, staging only by default) with live recovery graphs and pass or fail verdict
8. Audit Log	Every human or automated action with who, what, when and result; CSV export

11. Design System and Global Styles

The console uses **Tailwind CSS 4 and shadcn/ui**, as in the parent stack. The delivered **global.css** imports Tailwind, declares the brand tokens in an `@theme` block, maps them to the shadcn/ui variables (`--primary`, `--background`, `--ring` and others), and defines buttons, cards, badges and alerts. The full file is in Appendix A.

11.1 Brand colour palette

Navy 900 `#0B1F3A`	**Navy 800** `#12304F`	**Navy 700** `#1B4268`	**Primary 600** `#0F8B8D`
Primary 500 `#14A3A6`	**Cyan 400** `#22D3EE`	**Violet 500** `#7C3AED`	**Violet 400** `#A78BFA`
Success `#16A34A`	**Warning** `#F59E0B`	**Danger** `#DC2626`	**Info** `#2563EB`
Gray 50 `#F8FAFC`	**Gray 200** `#E2E8F0`	**Gray 500** `#64748B`	**Gray 900** `#0F172A`

11.2 Colour gradients

`--grad-primary`
#0F8B8D to #22D3EE, 135deg

`--grad-hero`
#0B1F3A - #12304F - #0F8B8D

`--grad-accent`
#7C3AED to #22D3EE

`--grad-success`
#16A34A to #4ADE80

`--grad-warning`
#F59E0B to #FCD34D

`--grad-danger`
#DC2626 to #F97316

`--grad-surface-dark`
#0F172A to #1E293B

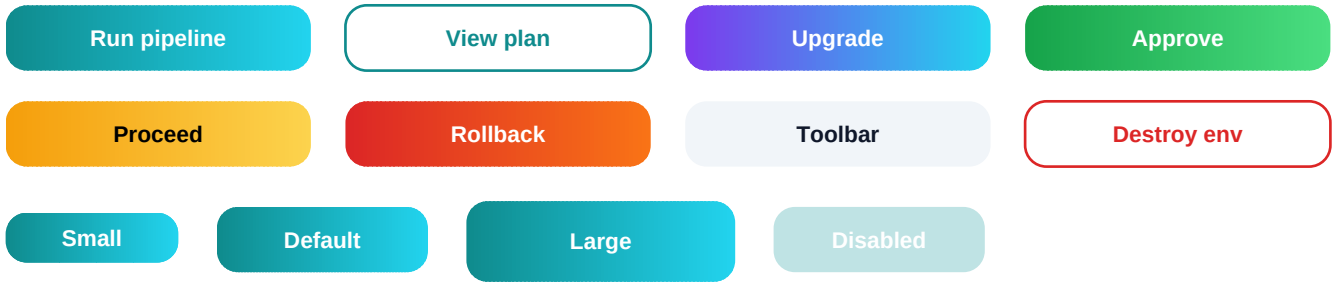
11.3 Typography

Role	Font	Size and weight
Headings	Inter	44 / 32 / 24 px, bold 700
Body	Inter	16 px, regular 400, line height 1.55
Small and labels	Inter	14 / 12 px, medium 500
Code, logs, digests	JetBrains Mono	14 px

11.4 Spacing, radius, shadow, motion

Token	Value
Spacing scale (4 px base)	4, 8, 12, 16, 24, 32, 48, 64
Radius	6 (small), 10 (medium), 16 (large), 999 (pill)
Shadow	sm, md, lg (soft navy-tinted)
Motion	120 ms fast, 200 ms standard, cubic-bezier(0.2, 0.8, 0.2, 1)

11.5 Buttons



Button	Class	Console use
Primary	btn btn-primary	Run experiment, Sync, Save
Secondary	btn btn-secondary	View details, Open in Grafana
Accent	btn btn-accent	Highlight actions such as Open trace
Success	btn btn-success	Promote canary, Approve
Warning	btn btn-warning	Pause rollout
Danger	btn btn-danger	Abort rollout, Rollback
Ghost	btn btn-ghost	Toolbar and table row actions
Outline danger	btn btn-outline-danger	Confirm-first destructive actions

Sizes: `btn-sm`, default, `btn-lg`, `btn-block`, `btn-icon`. States: hover lifts 1 px and brightens, focus shows a cyan ring, disabled is 50 percent opacity, loading shows a spinner and blocks clicks.

11.6 Status colours and components

Component	Style notes
Card	White surface, 16 px radius, soft shadow; hero card uses the hero gradient
Input	10 px radius, cyan focus ring, red border when invalid
Status badges	Pass (green), Warn (amber), Fail (red), Running (blue)
Severity chips	Critical red, High orange, Medium amber, Low blue
Health text	Healthy green (synced, passed), Degraded amber (progressing, paused), Down red (failed, aborted)
Alerts	Left colour bar with a light tinted background
Canary progress bar	Primary gradient fill with step markers at 10, 30, 60, 100
Charts (Recharts)	Primary, accent violet, success and warning colours; thresholds as dashed red lines

11.7 Accessibility

- Text contrast at least 4.5:1 (WCAG AA); never rely on colour alone, always pair with an icon or label.
- Visible keyboard focus on every interactive element.
- Reduced-motion preference respected.
- Dark theme is first-class, toggled with the data-theme attribute.

12. Week-by-Week Plan (10 weeks)

Week	Deliverable
1	Repos, OpenTofu and Ansible, k3s cluster on Azure (or Codespaces), ArgoCD bootstrapped
2	Demo microservices, Dockerfiles, Helm and Kustomize, deployed through ArgoCD
3	CI pipeline: tests, Trivy, Syft, Cosign, GitOps promotion
4	Kyverno policies, OpenBao and External Secrets, Istio mTLS, network policies
5	Prometheus, Grafana, Loki, Tempo, OpenTelemetry, alert rules
6	Argo Rollouts canary and analysis; autoscaling with HPA and KEDA
7	Remediation webhook, Falco, Chaos Mesh experiments with recorded results
8	OpenCost, Goldilocks, k6 load tests, cost comparison
9	Console backend and frontend (screens 1 to 8), Keycloak login
10	Documentation, architecture diagram, demo video, blog post, resume bullets

If you have only 4 to 6 weeks, drop Istio (use Argo Rollouts with Traefik or NGINX ingress), drop Tempo, and build only screens 2, 3 and 5.

13. Testing and Success Metrics

Metric	How to measure	Target
Commit to running in prod	GitHub Actions run time plus ArgoCD sync time	under 5 min
Insecure deployments blocked	Scripted attempts (unsigned, root, latest tag)	100% blocked
Automatic rollback time	Inject faulty release, time to full rollback	under 60 s
Pod failure recovery	Chaos Mesh pod kill under k6 load	under 30 s, error rate under 1%
Cost reduction	OpenCost, autoscaling on vs fixed replicas	30% or better
Availability under chaos	k6 checks during experiments	99.5% or better

Test tooling: pytest, Go test, Playwright for the console UI, and k6 for load.

14. Three-Minute Demo Script

1. Push a code change; show CI scanning, signing and the GitOps commit (30 s).
2. Watch ArgoCD sync and the canary shift traffic in the console (30 s).
3. Try deploying a root container and an unsigned image; show Kyverno blocking both (30 s).
4. Ship a faulty version; show the automatic rollback (40 s).
5. Kill a pod and a node under load; show recovery graphs (30 s).
6. Show the cost screen with the autoscaling savings (20 s).

15. Risks and How to Handle Them

Risk	Handling
Azure credit runs out	Deallocate VMs after each session, budget alerts, minimum stack first, record demos in short bursts
Regional vCPU quota or SKU unavailable	Try another region or a similar VM size; x86 sizes also work because images are multi-arch
Not a verified student (Track A unavailable)	Use Track B (Codespaces) and plan work in focused sessions within the monthly quota
Memory pressure	Istio, Tempo, Falco and the full Prometheus stack are heavy: start minimal, add one component at a time, set resource limits on the platform itself, and disable Istio and Tempo if needed
Free-offer terms change	Re-check the Azure for Students and Codespaces pages before starting
Version drift	Pin all chart and image versions and record them in VERSIONS.md
Scope creep	Finish a working end-to-end slice first (push, build, scan, deploy, monitor), then add canary, chaos and cost

16. Resume Impact

- Quantify results: commit-to-prod time, rollback time, recovery time, cost reduction.
- Public GitHub repos with a clear README, architecture diagram and short demo video; a blog post on design decisions and trade-offs.
- Everything as code (IaC, GitOps) so anyone can reproduce it.

- Map to certifications: CKA, CKS and Terraform Associate (OpenTofu skills transfer directly).

Suggested resume bullet: Designed and deployed a GitOps-based Kubernetes platform with automated CI/CD, Trivy and Falco security gates, canary releases and chaos-tested self-healing; reduced simulated infrastructure cost by 35% using KEDA autoscaling and OpenCost analytics.

Appendix A. global.css (full design-system stylesheet)

Also delivered as the separate file `global.css`. Import it once at the app root; Tailwind config reads the same tokens.

```
/* ========================================================================
   SentinelOps - global.css
   Design tokens + base styles + buttons + core components
   ======================================================================== */

/* ---------- 0. Tailwind CSS 4 + shadcn/ui bridge ---------- */
@import "tailwindcss";

@theme {
  --color-navy-900: #0B1F3A;
  --color-navy-800: #12304F;
  --color-brand: #0F8B8D;
  --color-brand-light: #14A3A6;
  --color-cyan-accent: #22D3EE;
  --color-violet-accent: #7C3AED;
  --color-success: #16A34A;
  --color-warning: #F59E0B;
  --color-danger: #DC2626;
  --color-info: #2563EB;
  --font-sans: "Inter", system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, monospace;
  --radius-card: 16px;
}

/* ---------- 1. Design tokens ---------- */
:root {
  /* shadcn/ui variables (components read these) */
  --background: #F8FAFC;
  --foreground: #0F172A;
  --primary: #0F8B8D;
  --primary-foreground: #FFFFFF;
  --secondary: #F1F5F9;
  --secondary-foreground: #0F172A;
  --destructive: #DC2626;
  --muted: #F1F5F9;
  --muted-foreground: #64748B;
  --ring: #22D3EE;
  --radius: 10px;

  /* Brand */
  --so-navy-900: #0B1F3A;
  --so-navy-800: #12304F;
  --so-navy-700: #1B4268;
  --so-primary-600: #0F8B8D;
  --so-primary-500: #14A3A6;
  --so-cyan-400: #22D3EE;
  --so-violet-500: #7C3AED;
  --so-violet-400: #A78BFA;

  /* Status */
  --so-success: #16A34A;
  --so-warning: #F59E0B;
  --so-danger: #DC2626;
  --so-info: #2563EB;

  /* Neutrals */
  --so-gray-50: #F8FAFC;
  --so-gray-100: #F1F5F9;
  --so-gray-200: #E2E8F0;
  --so-gray-300: #CBD5E1;
  --so-gray-500: #64748B;
  --so-gray-700: #334155;
  --so-gray-900: #0F172A;

  /* Gradients */
  --grad-primary: linear-gradient(135deg, #0F8B8D 0%, #22D3EE 100%);
  --grad-hero: linear-gradient(135deg, #0B1F3A 0%, #12304F 45%, #0F8B8D 100%);
  --grad-accent: linear-gradient(135deg, #7C3AED 0%, #22D3EE 100%);
```

```
  --grad-success: linear-gradient(135deg, #16A34A 0%, #4ADE80 100%);
  --grad-warning: linear-gradient(135deg, #F59E0B 0%, #FCD34D 100%);
  --grad-danger: linear-gradient(135deg, #DC2626 0%, #F97316 100%);
  --grad-surface-dark: linear-gradient(180deg, #0F172A 0%, #1E293B 100%);

  /* Semantic (light theme) */
  --bg: var(--so-gray-50);
  --surface: #FFFFFF;
  --surface-2: var(--so-gray-100);
  --border: var(--so-gray-200);
  --text: var(--so-gray-900);
  --text-muted: var(--so-gray-500);
  --link: var(--so-primary-600);
  --focus-ring: 0 0 0 3px rgba(34, 211, 238, 0.45);

  /* Typography */
  --font-sans: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, "SFMono-Regular", Menlo, monospace;
  --fs-xs: 0.75rem;
  --fs-sm: 0.875rem;
  --fs-md: 1rem;
  --fs-lg: 1.125rem;
  --fs-xl: 1.5rem;
  --fs-2xl: 2rem;
  --fs-3xl: 2.75rem;
  --lh-tight: 1.2;
  --lh-normal: 1.55;

  /* Spacing (4px base) */
  --sp-1: 4px;
  --sp-2: 8px;
  --sp-3: 12px;
  --sp-4: 16px;
  --sp-5: 24px;
  --sp-6: 32px;
  --sp-7: 48px;
  --sp-8: 64px;

  /* Radius, shadow, motion */
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 16px;
  --radius-pill: 999px;
  --shadow-sm: 0 1px 2px rgba(15, 23, 42, 0.08);
  --shadow-md: 0 4px 12px rgba(15, 23, 42, 0.10);
  --shadow-lg: 0 12px 32px rgba(15, 23, 42, 0.16);
  --ease: cubic-bezier(0.2, 0.8, 0.2, 1);
  --dur-fast: 120ms;
  --dur: 200ms;
}

/* ---------- 2. Dark theme ---------- */
[data-theme="dark"] {
  --background: #0B1220;
  --foreground: #E6EDF7;
  --secondary: #17233A;
  --muted: #17233A;
  --muted-foreground: #94A3B8;
  --bg: #0B1220;
  --surface: #111B2E;
  --surface-2: #17233A;
  --border: #243350;
  --text: #E6EDF7;
  --text-muted: #94A3B8;
  --link: var(--so-cyan-400);
  --shadow-md: 0 4px 12px rgba(0, 0, 0, 0.45);
}

/* ---------- 3. Base / reset ---------- */
*, *::before, *::after { box-sizing: border-box; }
html { -webkit-text-size-adjust: 100%; scroll-behavior: smooth; }
body {
  margin: 0;
  font-family: var(--font-sans);
  font-size: var(--fs-md);
```

```
  line-height: var(--lh-normal);
  color: var(--text);
  background: var(--bg);
}
h1, h2, h3, h4 { line-height: var(--lh-tight); margin: 0 0 var(--sp-3); font-weight: 700; }
h1 { font-size: var(--fs-3xl); }
h2 { font-size: var(--fs-2xl); }
h3 { font-size: var(--fs-xl); }
a { color: var(--link); text-decoration: none; }
a:hover { text-decoration: underline; }
code, pre { font-family: var(--font-mono); font-size: 0.9em; }
pre {
  background: var(--so-gray-900);
  color: #E2E8F0;
  padding: var(--sp-4);
  border-radius: var(--radius-md);
  overflow-x: auto;
}
:focus-visible { outline: none; box-shadow: var(--focus-ring); }

/* ---------- 4. Buttons ---------- */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: var(--sp-2);
  padding: 10px 18px;
  font: 600 var(--fs-sm)/1 var(--font-sans);
  border: 1px solid transparent;
  border-radius: var(--radius-md);
  cursor: pointer;
  user-select: none;
  transition: transform var(--dur-fast) var(--ease),
              box-shadow var(--dur) var(--ease),
              filter var(--dur) var(--ease),
              background var(--dur) var(--ease);
}
.btn:hover { transform: translateY(-1px); box-shadow: var(--shadow-md); }
.btn:active { transform: translateY(0); box-shadow: var(--shadow-sm); }
.btn:disabled, .btn[aria-disabled="true"] {
  opacity: 0.5; cursor: not-allowed; transform: none; box-shadow: none;
}

/* Variants */
.btn-primary   { background: var(--grad-primary); color: #fff; }
.btn-accent    { background: var(--grad-accent);  color: #fff; }
.btn-success   { background: var(--grad-success); color: #fff; }
.btn-warning   { background: var(--grad-warning); color: var(--so-gray-900); }
.btn-danger    { background: var(--grad-danger);  color: #fff; }
.btn-primary:hover, .btn-accent:hover, .btn-success:hover,
.btn-warning:hover, .btn-danger:hover { filter: brightness(1.08); }

.btn-secondary {
  background: var(--surface);
  color: var(--so-primary-600);
  border-color: var(--so-primary-600);
}
.btn-secondary:hover { background: rgba(15, 139, 141, 0.08); }

.btn-ghost { background: transparent; color: var(--text); }
.btn-ghost:hover { background: var(--surface-2); box-shadow: none; }

.btn-outline-danger {
  background: transparent;
  color: var(--so-danger);
  border-color: var(--so-danger);
}
.btn-outline-danger:hover { background: rgba(220, 38, 38, 0.08); }

/* Sizes */
.btn-sm { padding: 6px 12px; font-size: var(--fs-xs); border-radius: var(--radius-sm); }
.btn-lg { padding: 14px 26px; font-size: var(--fs-md); border-radius: var(--radius-lg); }
.btn-block { width: 100%; }
.btn-icon { padding: 10px; aspect-ratio: 1; }
```

```
/* Loading state */
.btn.is-loading { position: relative; color: transparent; pointer-events: none; }
.btn.is-loading::after {
  content: "";
  position: absolute;
  width: 16px; height: 16px;
  border: 2px solid rgba(255, 255, 255, 0.4);
  border-top-color: #fff;
  border-radius: 50%;
  animation: so-spin 0.7s linear infinite;
}
@keyframes so-spin { to { transform: rotate(360deg); } }

/* ---------- 5. Cards, inputs, badges, alerts ---------- */
.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: var(--sp-5);
  box-shadow: var(--shadow-sm);
}
.card-hero { background: var(--grad-hero); color: #fff; border: 0; }

.input {
  width: 100%;
  padding: 10px 12px;
  font: var(--fs-sm) var(--font-sans);
  color: var(--text);
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
}
.input:focus { border-color: var(--so-cyan-400); box-shadow: var(--focus-ring); }
.input[aria-invalid="true"] { border-color: var(--so-danger); }

.badge {
  display: inline-block;
  padding: 2px 10px;
  font: 600 var(--fs-xs)/1.6 var(--font-sans);
  border-radius: var(--radius-pill);
}
.badge-pass     { background: rgba(22, 163, 74, 0.15);  color: var(--so-success); }
.badge-warn     { background: rgba(245, 158, 11, 0.18); color: #B45309; }
.badge-fail     { background: rgba(220, 38, 38, 0.15);  color: var(--so-danger); }
.badge-running  { background: rgba(37, 99, 235, 0.15);  color: var(--so-info); }
.badge-critical { background: var(--so-danger); color: #fff; }
.badge-high     { background: #EA580C; color: #fff; }
.badge-medium   { background: var(--so-warning); color: var(--so-gray-900); }
.badge-low      { background: var(--so-info); color: #fff; }

.alert {
  padding: var(--sp-3) var(--sp-4);
  border-left: 4px solid var(--so-info);
  border-radius: var(--radius-sm);
  background: rgba(37, 99, 235, 0.08);
}
.alert-danger  { border-color: var(--so-danger);  background: rgba(220, 38, 38, 0.08); }
.alert-success { border-color: var(--so-success); background: rgba(22, 163, 74, 0.08); }
.alert-warning { border-color: var(--so-warning); background: rgba(245, 158, 11, 0.10); }

/* ---------- 6. Health / status colours (nodes, rollouts, experiments) ---------- */
.status-healthy  { color: var(--so-success); }  /* synced, passed, ready */
.status-degraded { color: var(--so-warning); }  /* progressing, paused   */
.status-down     { color: var(--so-danger); }   /* failed, aborted       */

/* ---------- 7. Utilities ---------- */
.text-gradient {
  background: var(--grad-primary);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
.stack > * + * { margin-top: var(--sp-4); }
.row { display: flex; gap: var(--sp-3); align-items: center; }
.sr-only {
```

```
  position: absolute; width: 1px; height: 1px; overflow: hidden;
  clip: rect(0 0 0 0); white-space: nowrap;
}
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
```